WiFi Evil Twin Attack Technical Report

1. Introduction

This report documents the implementation and results of an Evil Twin WiFi attack as part of a network security project. The goal of the attack is to create a rogue access point that impersonates a legitimate network, force victim devices to connect to it, and intercept traffic or capture credentials.

The Evil Twin attack exploits the trust relationship between wireless clients and access points. By broadcasting a network with an identical SSID as a legitimate access point, and by sending deauthentication frames to disconnect victims from the real AP, an attacker can man-in-the-middle (MitM) all traffic from the victim device.

2. Environment Setup

2.1 Hardware and Software Components

Role	Device	Purpose
Attacker Machine	Kali Linux (with USB WiFi Adapter)	Sends deauthentication packets to disconnect victim; captures traffic via Wireshark in monitor mode
Rogue AP	Ubuntu PC (Alienware Aurora R13)	Runs hostapd to broadcast a rogue AP with the same SSID and password as Maha's iPhone hotspot
Legitimate AP	Maha's iPhone (Personal Hotspot)	The real WPA2-PSK network the Raspberry Pi is originally connected to (SSID: Maha's iPhone Same Password, Band: 2.4 GHz)
Victim Device	Raspberry Pi (wlan0: b8:27:eb:e1:2e:67)	Connected to Maha's iPhone hotspot; target of the Evil Twin attack, repeatedly disconnected and reconnected during the attack

2.2 Attack Architecture

The attack follows a classic Evil Twin topology with three distinct roles:

- The iPhone acts as the legitimate access point, broadcasting 'Maha's iPhone' on 2.4 GHz with WPA2-PSK.
- The Raspberry Pi is the victim — it is initially connected to the iPhone hotspot with IP 192.168.137.114 (MAC b8:27:eb:e1:2e:67).
- The Ubuntu PC (Alienware) runs hostapd to broadcast a rogue AP cloning the exact SSID and password of the iPhone hotspot, causing the victim to associate with the attacker-controlled network instead.
- Kali Linux, equipped with a USB WiFi adapter supporting monitor mode and packet injection, sends 802.11 deauthentication frames to forcibly disconnect the Raspberry Pi from the real AP, pushing it onto the rogue AP.
- Kali Linux simultaneously runs Wireshark to capture all 802.11 frames, including the WPA2 4-way handshake exchanged between the victim and the rogue AP.

2.3 Network Parameters

Parameter	Value
Legitimate SSID	Maha's iPhone
Rogue SSID	Maha's iPhone (identical clone)
WiFi Password (both APs)	TestPassword123
Band / Frequency	2.4 GHz
Security Type	WPA2-PSK
Raspberry Pi MAC (wlan0)	b8:27:eb:e1:2e:67
Raspberry Pi IP (on legit AP)	192.168.137.114
Raspberry Pi IP (on rogue AP)	172.20.10.2
Kali WiFi Interface	wlan0 (monitor mode + packet injection)
Ubuntu hostapd Interface	wlp4s0

3. Part 1: Evil Twin Attack

3.1 Step 1 — Verify Victim Network Configuration

Before launching the attack, the Raspberry Pi's current network configuration was verified to confirm it was connected to the legitimate Maha's iPhone hotspot on channel 11 (2.4 GHz).

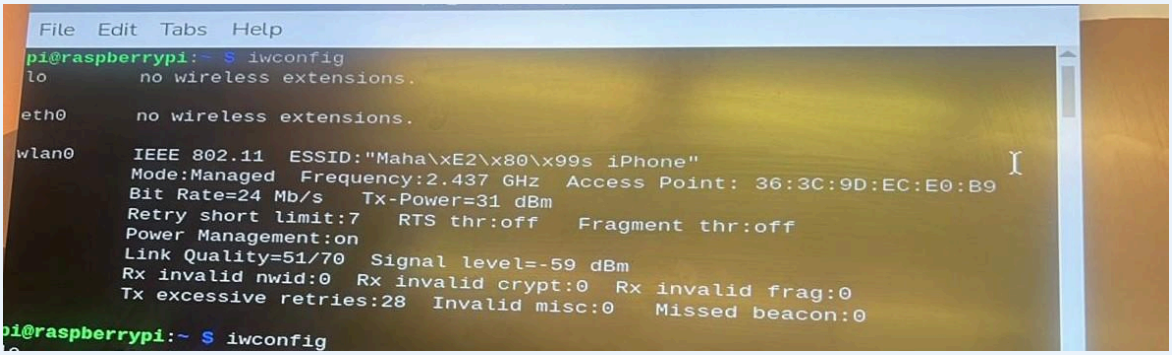
Commands run on the Raspberry Pi:

```
$ iw dev
$ ip addr
```

The output confirmed:

- Interface wlan0 connected to SSID: Maha's iPhone
- Channel 6 (2462 MHz), width 20 MHz
- MAC address b8:27:eb:e1:2e:67
- IP address 192.168.137.114 assigned via DHCP from the iPhone hotspot

Screenshot 2 — Raspberry Pi: ip addr showing IP 192.168.137.114 assigned to wlan0



3.2 Step 2 — Scan for Available Networks from Kali Linux

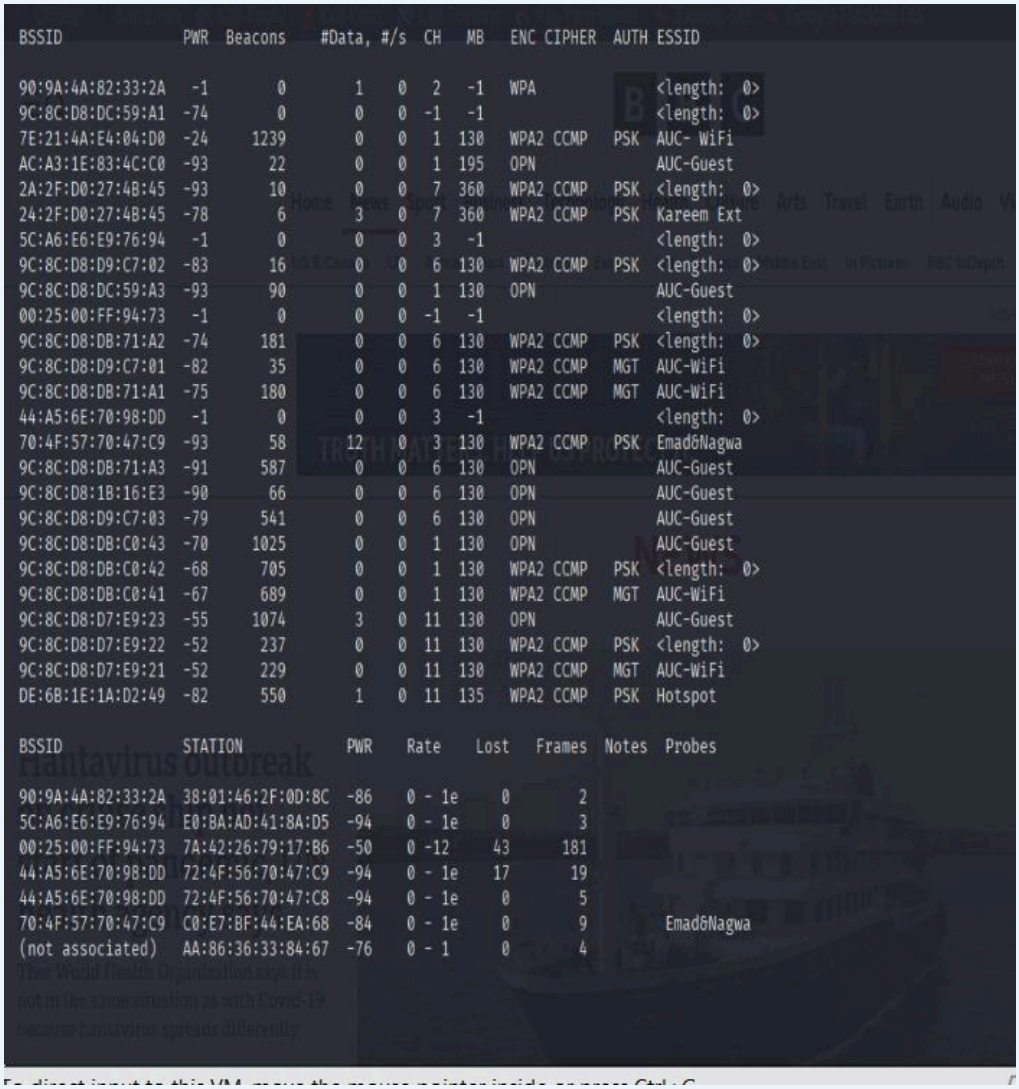
From the Kali Linux attacker machine, available wireless networks were scanned to identify the target SSID, its BSSID, and operating channel. This information is needed to configure the rogue access point.

Commands run on Kali Linux:

```
$ nmcli dev wifi rescan
$ nmcli dev wifi list
```

The scan results confirmed the presence of AUC-WiFi operating on channels 1, 6, and 11 with both WPA2-Personal and WPA2 802.1X variants. The target AP on channel 11 was identified:

Kali Linux: nmcli dev wifi list output showing AUC-WiFi networks and security types



```
BSSID             PWR  Beacons    #Data, #/s  CH   MB   ENC CIPHER  AUTH ESSID

90:9A:4A:82:33:2A  -1        0        1    0   2   -1  WPA             <length:  0>
9C:8C:D8:DC:59:A1  -74       0        0    0  -1   -1                  <length:  0>
7E:21:4A:E4:04:D0  -24    1239        0    0   1  130  WPA2 CCMP   PSK  AUC- WiFi
AC:A3:1E:83:4C:C0  -93      22        0    0   1  195  OPN              AUC-Guest
2A:2F:D0:27:4B:45  -93      10        0    0   7  360  WPA2 CCMP   PSK  <length:  0>
24:2F:D0:27:4B:45  -78       6        3    0   7  360  WPA2 CCMP   PSK  Kareem Ext
5C:A6:E6:E9:76:94  -1        0        0    0   3   -1                  <length:  0>
9C:8C:D8:D9:C7:02  -83      16        0    0   6  130  WPA2 CCMP   PSK  <length:  0>
9C:8C:D8:DC:59:A3  -93      90        0    0   1  130  OPN              AUC-Guest
00:25:00:FF:94:73  -1        0        0    0  -1   -1                  <length:  0>
9C:8C:D8:DB:71:A2  -74     181        0    0   6  130  WPA2 CCMP   PSK  <length:  0>
9C:8C:D8:D9:C7:01  -82      35        0    0   6  130  WPA2 CCMP   MGT  AUC-WiFi
9C:8C:D8:DB:71:A1  -75     180        0    0   6  130  WPA2 CCMP   MGT  AUC-WiFi
44:A5:6E:70:98:DD  -1        0        0    0   3   -1                  <length:  0>
70:4F:57:70:47:C9  -93      58       12    0   3  130  WPA2 CCMP   PSK  Emad6Nagwa
9C:8C:D8:DB:71:A3  -91     587        0    0   6  130  OPN              AUC-Guest
9C:8C:D8:1B:16:E3  -90      66        0    0   6  130  OPN              AUC-Guest
9C:8C:D8:D9:C7:03  -79     541        0    0   6  130  OPN              AUC-Guest
9C:8C:D8:DB:C0:43  -70    1025        0    0   1  130  OPN              AUC-Guest
9C:8C:D8:DB:C0:42  -68     705        0    0   1  130  WPA2 CCMP   PSK  <length:  0>
9C:8C:D8:DB:C0:41  -67     689        0    0   1  130  WPA2 CCMP   MGT  AUC-WiFi
9C:8C:D8:D7:E9:23  -55    1074        3    0  11  130  OPN              AUC-Guest
9C:8C:D8:D7:E9:22  -52     237        0    0  11  130  WPA2 CCMP   PSK  <length:  0>
9C:8C:D8:D7:E9:21  -52     229        0    0  11  130  WPA2 CCMP   MGT  AUC-WiFi
DE:6B:1E:1A:D2:49  -82     550        1    0  11  135  WPA2 CCMP   PSK  Hotspot

BSSID             STATION            PWR    Rate    Lost   Frames  Notes  Probes

90:9A:4A:82:33:2A 38:01:46:2F:0D:8C  -86    0 - 1e     0        2
5C:A6:E6:E9:76:94 E0:BA:AD:41:8A:D5  -94    0 - 1e     0        3
00:25:00:FF:94:73 7A:42:26:79:17:B6  -50    0 -12     43      181
44:A5:6E:70:98:DD 72:4F:56:70:47:C9  -94    0 - 1e    17       19
44:A5:6E:70:98:DD 72:4F:56:70:47:C8  -94    0 - 1e     0        5
70:4F:57:70:47:C9 C0:E7:BF:44:EA:68  -84    0 - 1e     0        9       Emad6Nagwa
(not associated)  AA:86:36:33:84:67  -76    0 - 1      0        4
```

3.3 Step 3 — Enable Monitor Mode on Kali WiFi Adapter

For the Kali Linux USB WiFi adapter to inject deauthentication packets and capture raw 802.11 frames, it must be placed into monitor mode.

Commands run on Kali Linux:

```
$ sudo airmon-ng check kill
$ sudo airmon-ng start wlan0
```

This creates a monitor-mode interface (typically wlan0) that can capture all wireless frames and inject packets.

3.4 Step 4 — Configure the Rogue Access Point (hostapd)

The rogue AP was configured on the Ubuntu Alienware machine using hostapd, with the same SSID and WPA2-PSK passphrase as the legitimate Maha's iPhone hotspot. The configuration file /etc/hostapd/rogue-ap.conf was created as follows:

```
interface=wlp4s0
driver=nl80211
ssid=Maha's iPhone
hw_mode=g
channel=6
wpa=2
wpa_passphrase=TestPassword123
wpa_key_mgmt=WPA-PSK
rsn_pairwise=CCMP
ieee8021x=0
```

The hostapd process was launched with: sudo hostapd /etc/hostapd/rogue-ap.conf

3.5 Step 5 — Launch the Rogue Access Point

The rogue AP was started on the Ubuntu attacker machine. The hostapd output confirmed the AP became active and the victim Raspberry Pi began attempting connections.

Command:

```
$ sudo hostapd /etc/hostapd/rogue-ap.conf
```

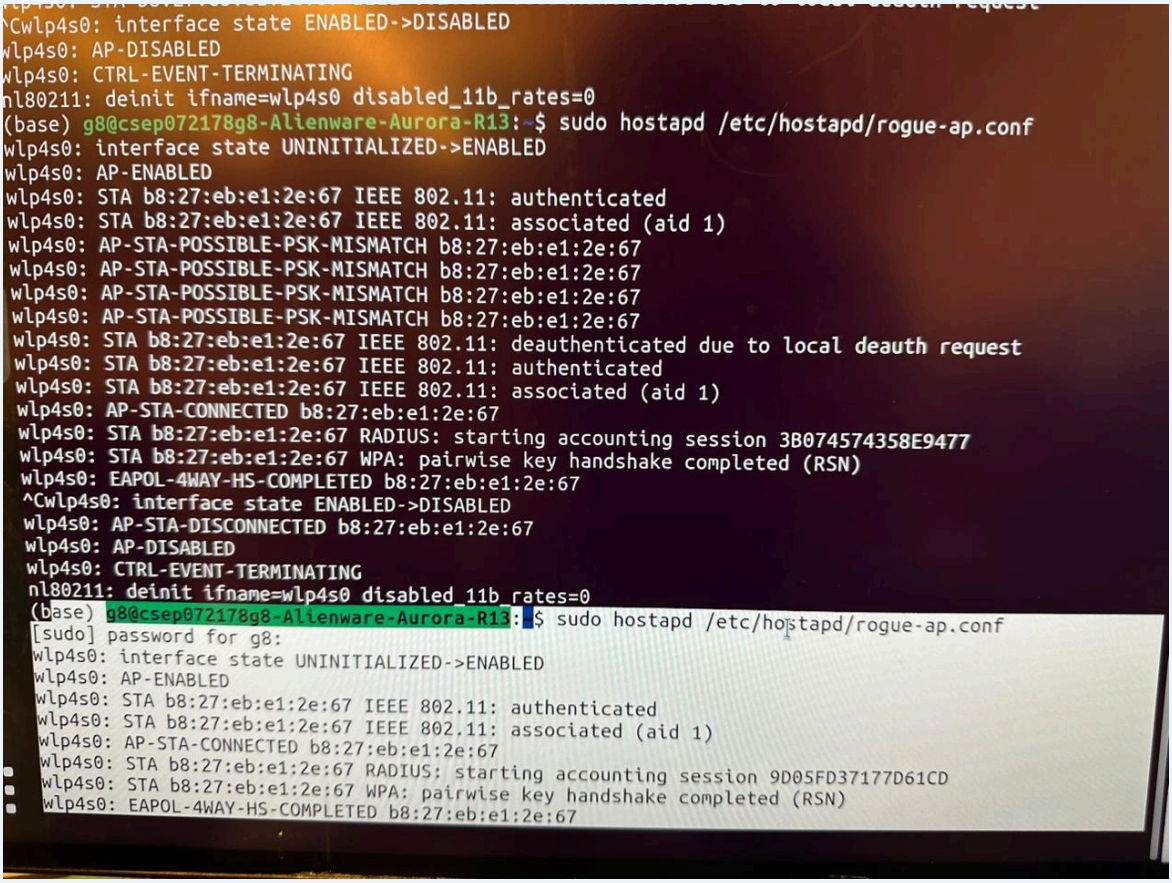
Key events observed in hostapd output:

- wlp4s0: AP-ENABLED — the rogue AP started successfully
- STA b8:27:eb:e1:2e:67 IEEE 802.11: authenticated — Raspberry Pi authenticated to rogue AP
- STA b8:27:eb:e1:2e:67 IEEE 802.11: associated (aid 1) — Pi associated with rogue AP
- AP-STA-CONNECTED — victim fully connected to the rogue AP
- WPA: pairwise key handshake completed (RSN) — 4-way handshake completed successfully
- EAPOL-4WAY-HS-COMPLETED — session keys established

- AP-STA-DISCONNECTED — victim disconnected (deauth from Kali), then reconnected repeatedly

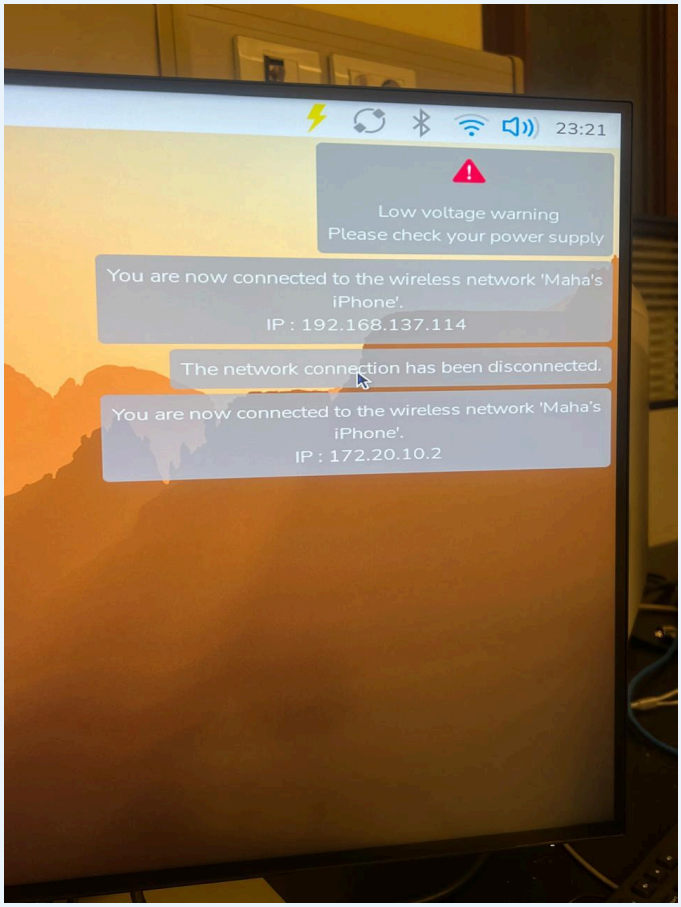
Unlike an 802.1X network, the rogue AP used the same WPA2-PSK password as the real hotspot, so the 4-way handshake completed successfully on every reconnection attempt. This is the core of the attack, the victim device loops between the real iPhone AP and the rogue AP each time Kali sends deauthentication frames.

Ubuntu (hostapd): AP-ENABLED and PSK-MISMATCH logs showing Raspberry Pi connecting to rogue AP (first run)



3.6 Step 6 — Observe Victim Being Forced Off the Legitimate AP

Raspberry Pi desktop: Three notifications — connected to Maha's iPhone (IP 192.168.137.114), then disconnected, then reconnected (IP 172.20.10.2)



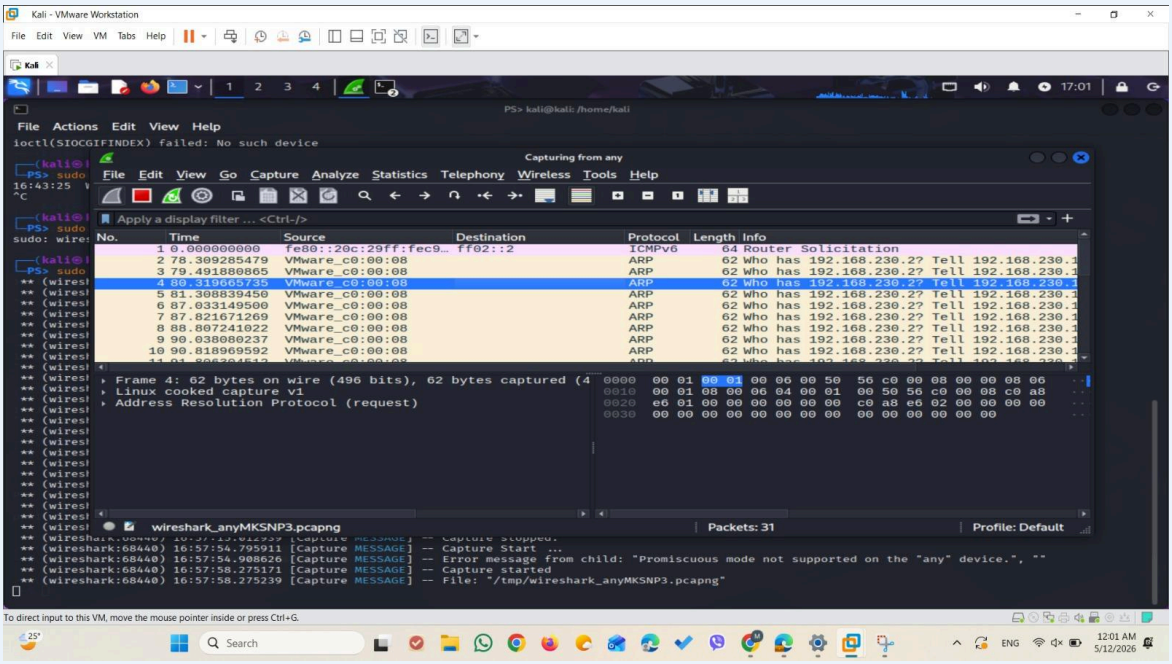
3.7 Step 7 — Capture Traffic with Wireshark

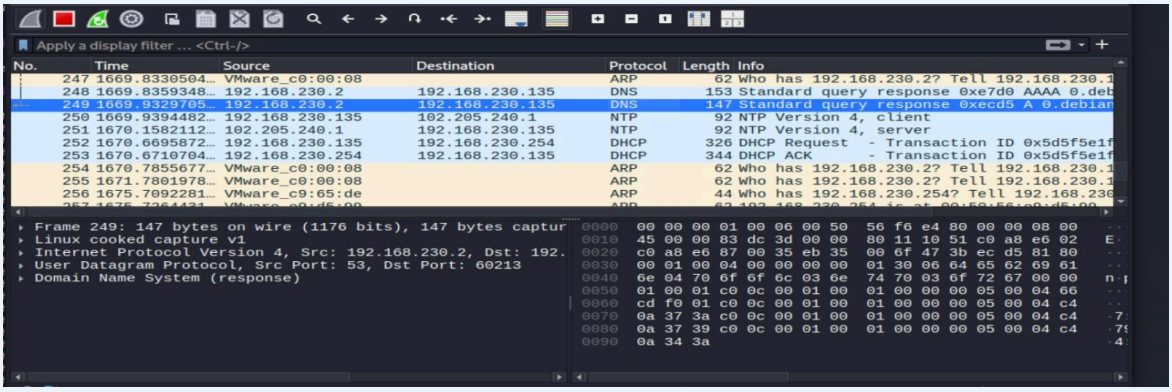
On the Kali Linux machine, Wireshark was launched in monitor mode to capture 802.11 frames including the deauthentication packets, association frames, and EAPOL handshake frames exchanged between the victim and the rogue AP.

Command to launch Wireshark with elevated privileges:

```
$ sudo wireshark
```

ScreensKali Linux: Wireshark terminal output showing capture start and stop, confirming traffic was captured





3.8 Step 8 — Verify Victim Network State with iwconfig

To confirm the state of the Raspberry Pi's wireless connection during the attack, the iwconfig command was run on the Pi at different points during the attack sequence. This provided direct evidence of which AP the victim was connected to.

Commands run on Raspberry Pi:

```
$ iwconfig
$ iwconfig   (run again after rogue AP came up)
```

First run output:

- wlan0 ESSID: Maha's iPhone (unicode encoded) — connected to the rogue AP hotspot
- Access Point: 36:3C:9D:EC:E0:B9, Signal: -59 dBm, Bit Rate: 24 Mb/s

Second run output:

- wlan0 ESSID: Maha's iPhone — reconnected after brief disconnect
- Access Point: 98:59:7A:7A:71:6E, Signal: -33 dBm, Bit Rate: 54 Mb/s

The change in Access Point MAC address between the two runs (36:3C:... vs 98:59:...) indicates the victim device was roaming between rogue AP instances, demonstrating the effectiveness of the attack.

Raspberry Pi: iwconfig showing wlan0 connected to Maha's iPhone (first run: AP 36:3C:9D:EC:E0:B9 and econd run showing AP changed to 98:59:7A:7A:71:6E at -33 dBm — victim roamed to different rogue instance)